

Unit I :: Introduction to a Programming Environment

☞ **SEC-01-PHY-91-R-03** ☞

Dr. Parag Bhattacharya
Department of Physics
Rangapara College (Autonomous)

Syllabus

Unit I: Introduction to a Programming Environment

Software installation and start-up; locating and launching the application, exploring the graphical interface of the software: console, script editor, variable browser; understanding the workspace: current directory, command history, working directory settings; using built-in help and documentation features; basic command execution: arithmetic expressions, variable assignments, managing output display; data types and variables: numeric types (integers, floats), strings, booleans; variable naming conventions; operators and expressions: arithmetic, relational, logical operators; operator precedence and associativity; code structure and readability: using comments, breaking long expressions, line continuation symbols; saving and executing scripts, running code from the script editor or the command window.

1 Setting up the Scilab software

1.1 Installing Scilab

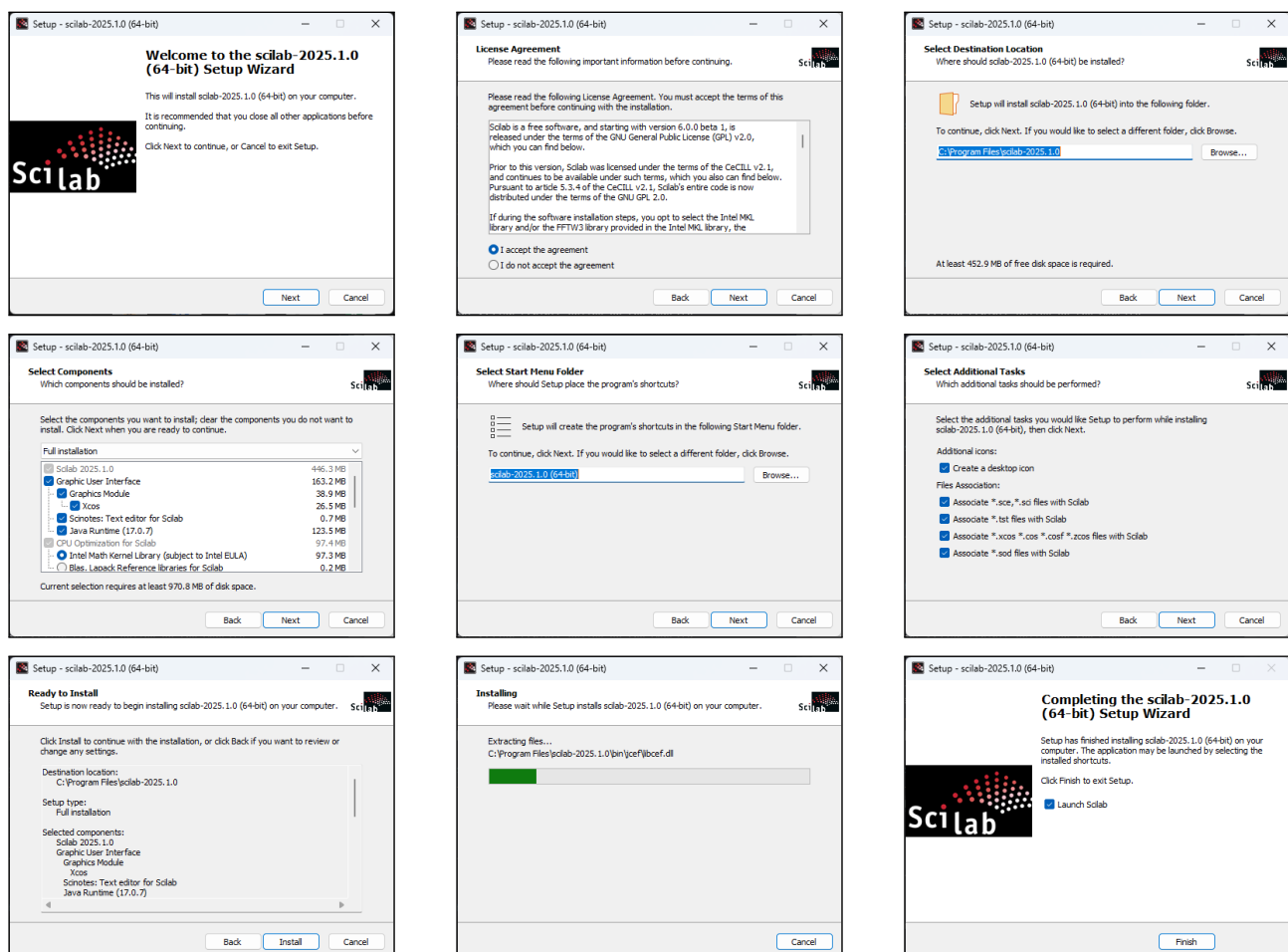
I. Visit the download page of Scilab available at: <https://www.scilab.org/download/>



Figure 1: The webpage for downloading the latest version of Scilab

II. Click on the relevant link to download Scilab for Windows, Linux or MacOS.

III. Navigate to the folder containing the downloaded file and install the software on the PC. Select 'English' as the preferred language the then click on 'OK' to proceed with the rest of the installation process as illustrated below:



1.2 Scilab on Cloud

The *National Virtual Library of India*¹ maintains an online version of Scilab on the Garuda Cloud. This is known as *Scilab on Cloud* and is available at: <https://cloud.scilab.in/>

The online interface of *Scilab on Cloud* is displayed in figure 3.

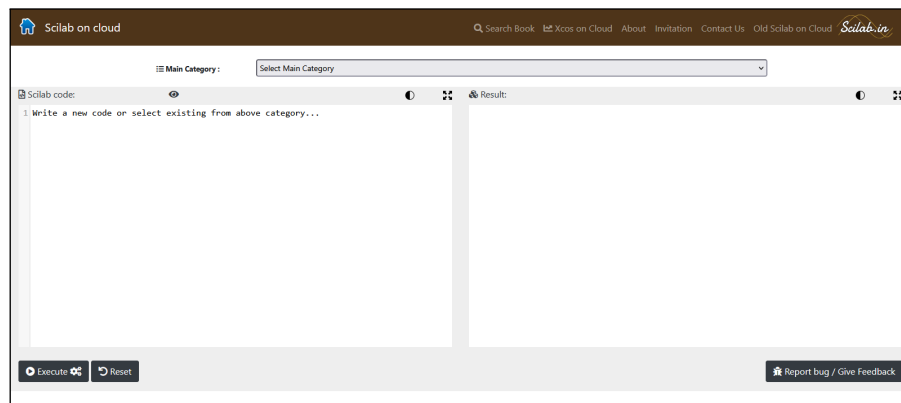


Figure 3: Scilab on Cloud

One can type the Scilab code on the left panel, then click on 'Execute' to view the output on the right panel.

1.3 Launching Scilab

One can find the Scilab icon on the desktop or in the list of installed applications on one's PC. Simply clicking on the icon launches Scilab with its default GUI which is displayed in figure 4 below.

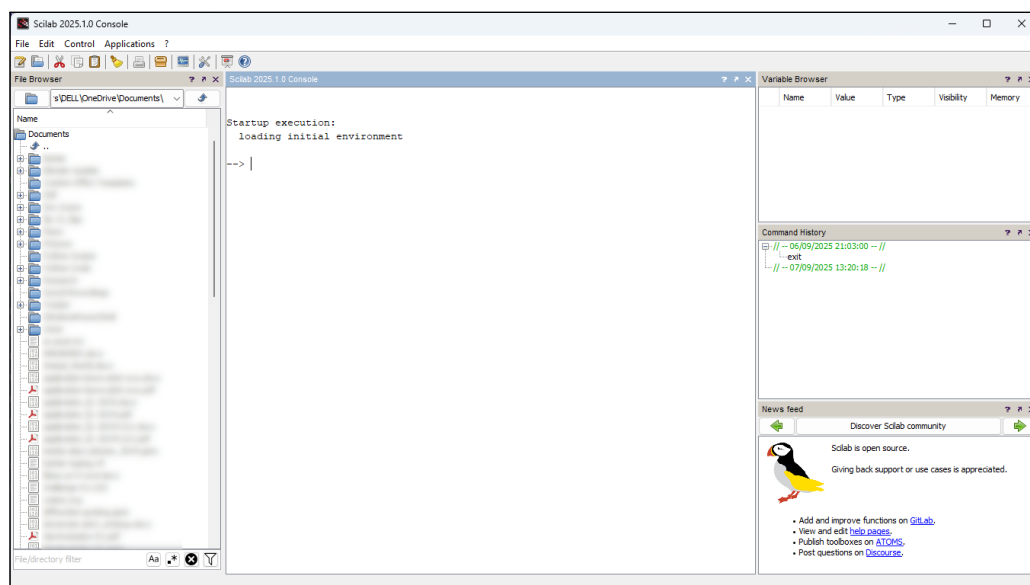


Figure 4: Default graphical user interface (GUI) of Scilab

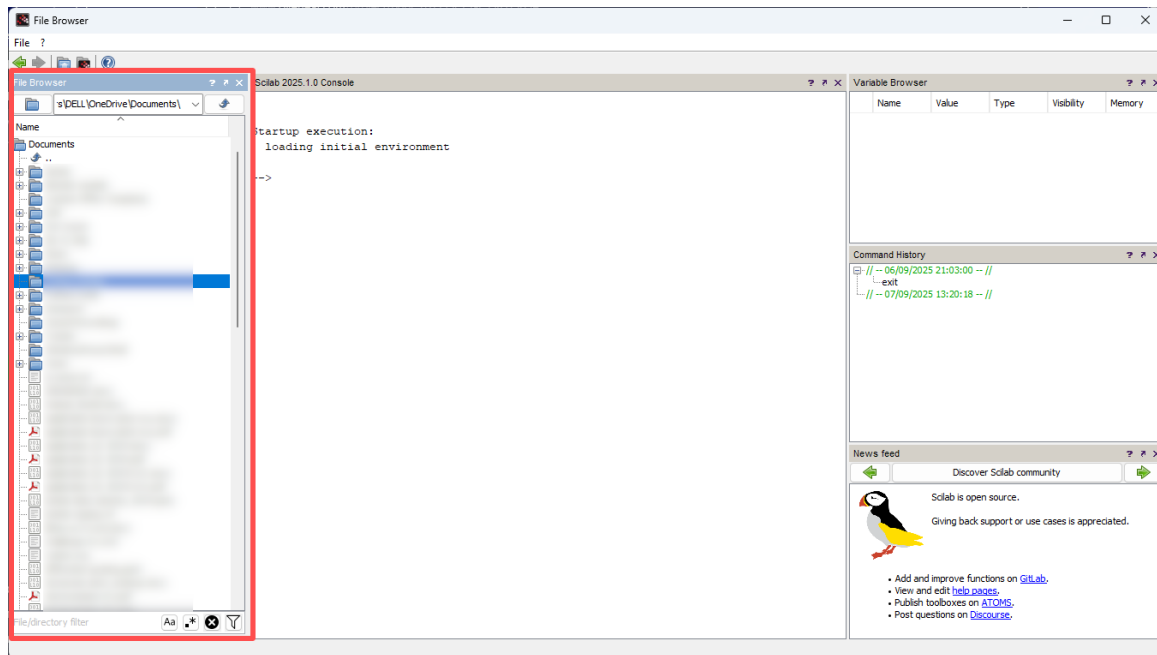
¹<https://www.rrr1f.gov.in/NML/NVLI.aspx>

2 Understanding the GUI of Scilab

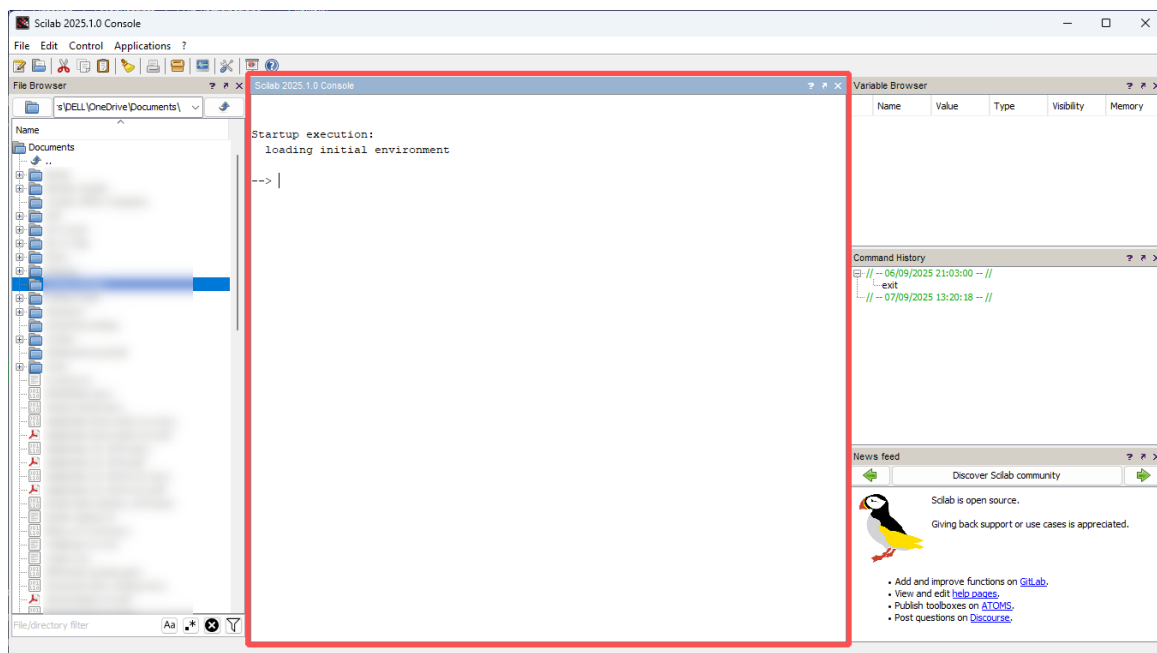
2.1 Panes and layout

The layout of Scilab's GUI is divided into five distinct panes, each of which has its own unique functionality.

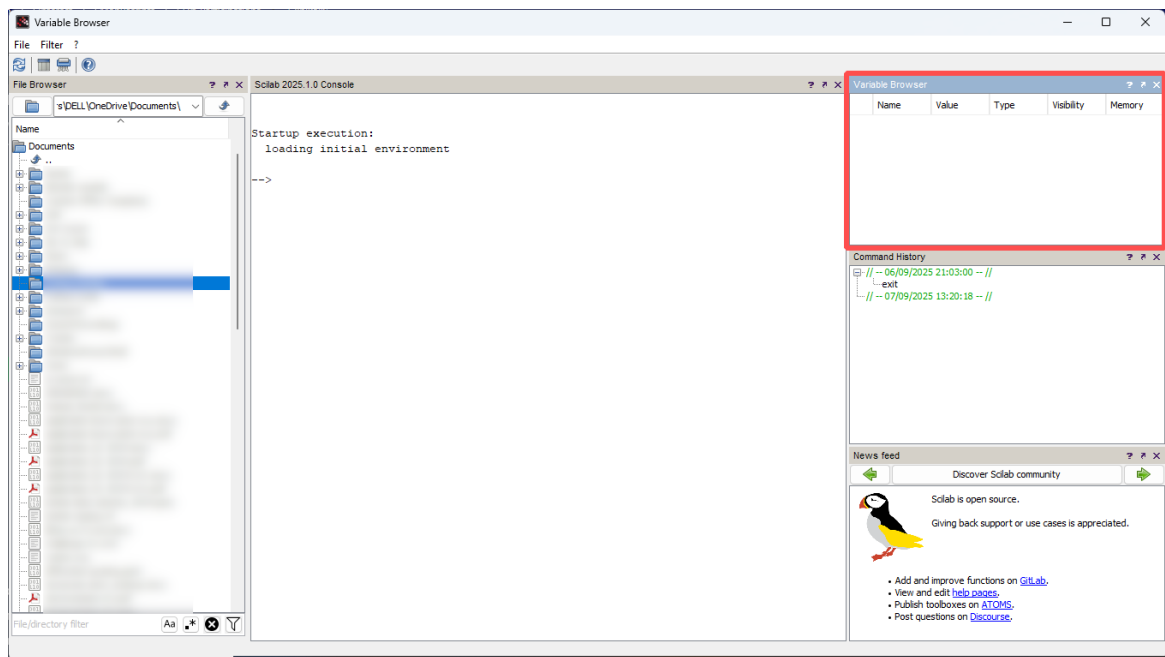
1. *File browser* – allows one to browse through the directory tree to easily navigate to the relevant Scilab script file or folder containing the same.



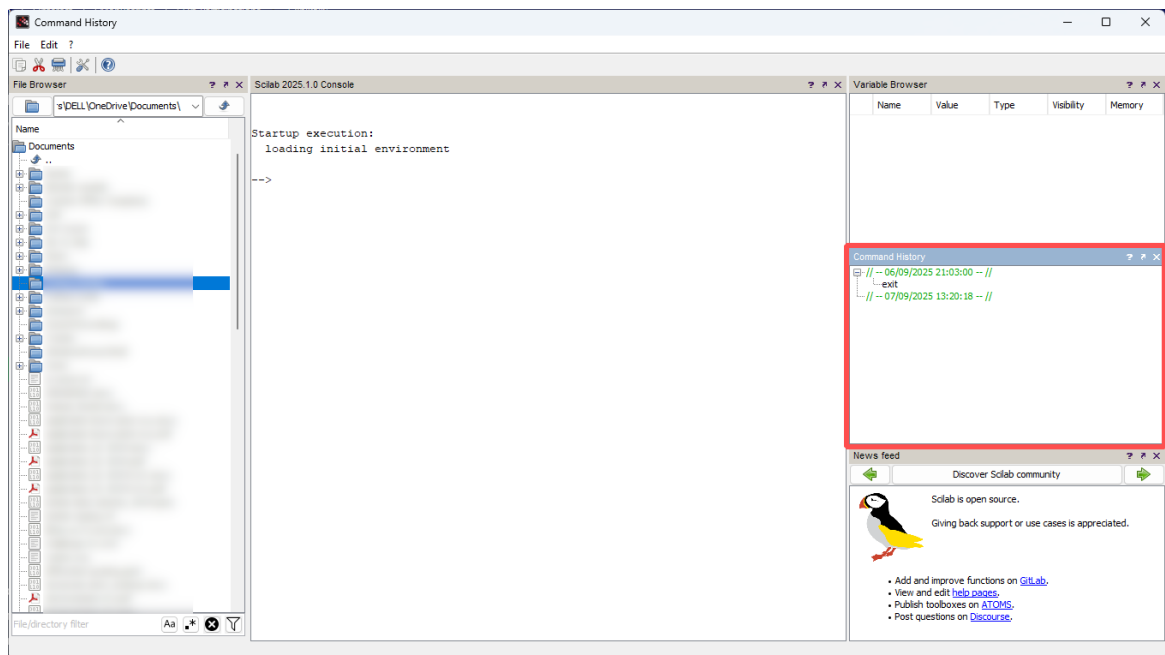
2. *Console window* – allows one to type and execute Scilab commands, as well as to view the output.



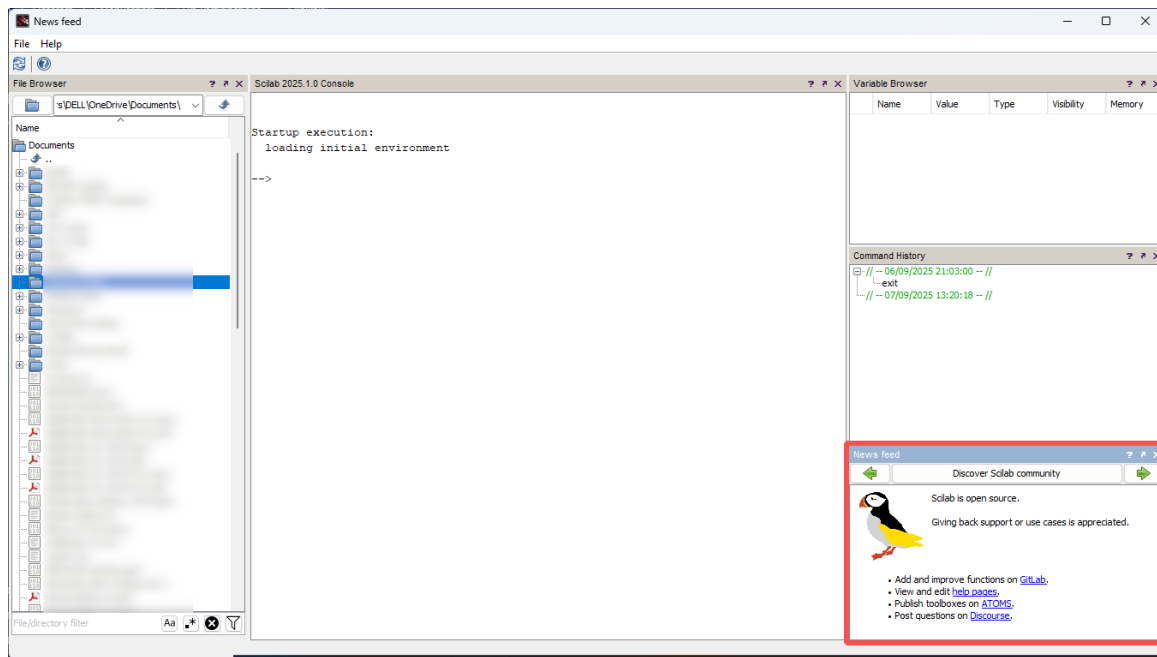
3. *Variable browser* – displays all the variable that have been created by Scilab and are currently active in memory locations.



4. *Command history* – display a chronological order of all the Scilab commands executed by the user, with the most recent command being at the top.



5. *News feed* – provides Scilab news and an interface to interact with the online Scilab community.



2.2 The Workspace

The Scilab workspace consists of the following components:

1. The console window

- Docked on the GUI by default.
- One can perform numerical calculations right from the console and view the output as soon as the computations are complete.
- One can find the current directory that one is working on by using the command `pwd` (which stands for *present working directory*) on the console window.
- The Scilab console presents a full-featured, interactive, command-line REPL (*read-evaluate-print-loop*) shell.

2. The variable browser

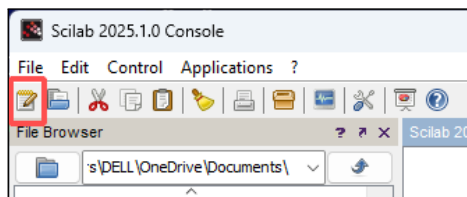
- Docked on the GUI by default.
- One can view the names of the variables, the value they store as well as the memory they are allocated.

3. The command history

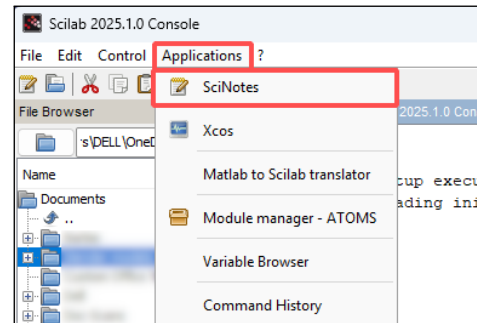
- Docked on the GUI by default.
- One can click on any of the earlier commands from the command history to re-run the same without having to type the command all over again.

4. **The SciNotes editor:** Running multiple lines of command is not possible on the console window. Also, the console window does not allow one to save a program such that it can be executed later. For these reasons, Scilab provides its own editor, known as SciNotes, that allow the user to write and save multi-line programs that can be executed at any point of time on any other PC with Scilab installed.

- Not docked on the GUI by default.
- To open SciNotes, one can either click on its icon on the taskbar (as shown in figure 5a), or one can use the main menu by clicking Applications → SciNotes (as shown in figure 5b).



(a) Opening SciNotes using the taskbar icon



(b) Opening SciNotes using the main menu

Figure 5: Two ways to open SciNotes

5. **The graphic window:** The graphic window is used for displaying any graphical output, such as the plot resulting from a numerical computation or a simulation result.

- Not docked on the GUI by default.
- A graphical window opens up automatically when making any plot.
- A graphical window may be opened by running any of the plot commands in Scilab either from the console window or by including it as a line in a Scilab program written using SciNotes.
- One can view a generic sample plot using a graphical window by simply typing `plot` on the console window, as illustrated in figure 6.

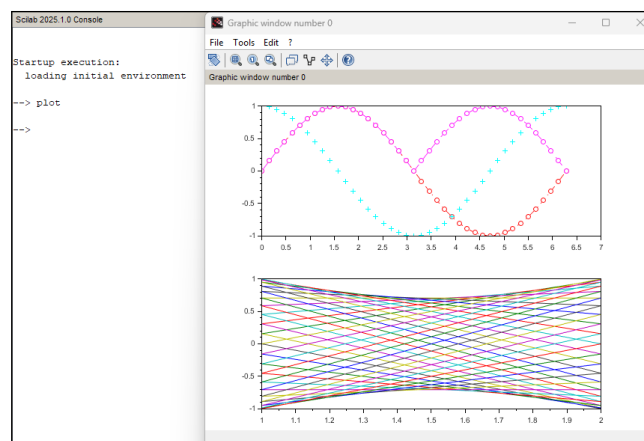


Figure 6: Generic sample plot on a graphical window

2.3 Documentation and help

Scilab provides plenty of resources in the form of documentation that can be easily accessed from the main menu by clicking ?. We are primarily interested in two of the sub-menu items, namely Scilab Help (which can be directly accessed by pressing the F1 key) and Scilab Demonstrations, as illustrated in figure 7.

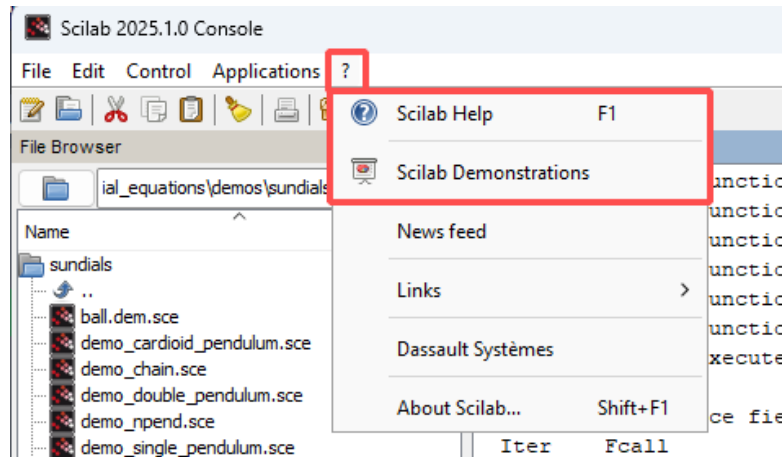


Figure 7: Documentation in Scilab

- **Scilab Help:** The Scilab Help Browser window provides detailed documentation on all the features and functions provided in various toolboxes that come bundled with Scilab.

One can either navigate to the exact topic using the hierarchical directory tree structure, or one may search for the specific topic of interest.

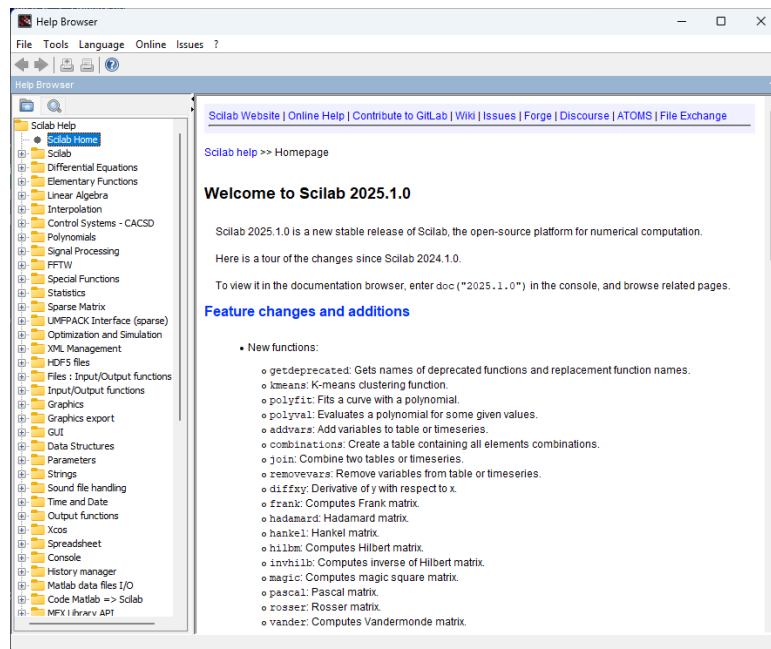


Figure 8: Scilab help browser window

- **Scilab Demonstrations:** Scilab provides a collection of demonstration scripts, which are available on the demonstrations menu. In figure 9, an example is displayed as to how one

can access a particular demonstration, in this case by clicking Simulation → ODE's → Lorenz equation from the demonstrations menu. This would open up a graphic window where the demonstration is performed. One can also view the underlying Scilab code by clicking on -- View Code -- on the main menu of the same graphic window.

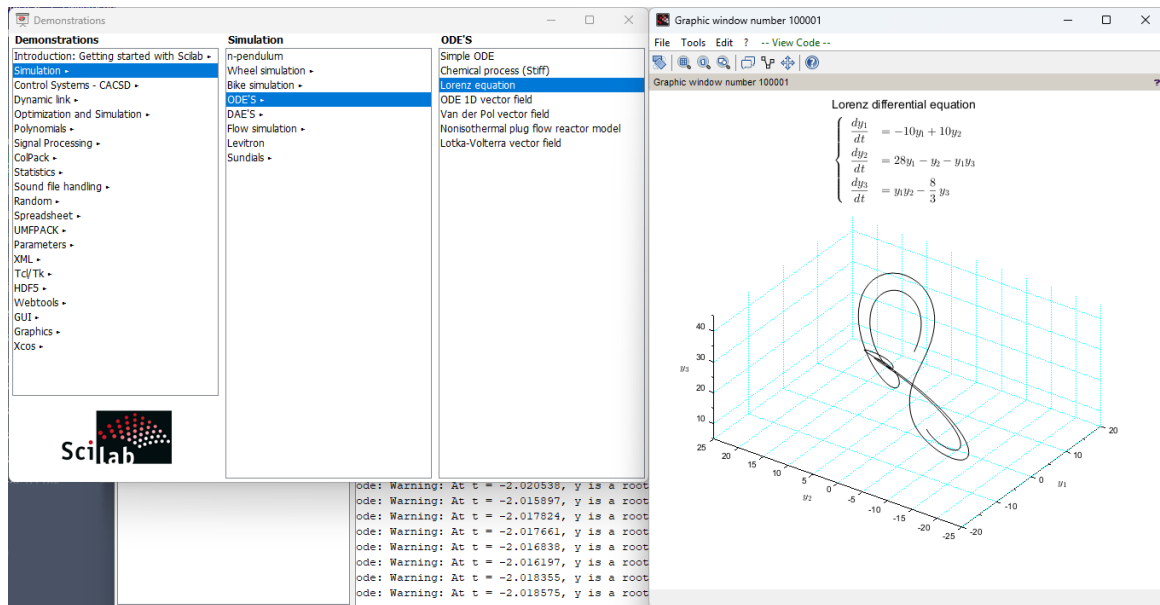


Figure 9: Scilab help browser window

3 Basic command execution

3.1 Arithmetic expressions

Scilab can be used as a simple calculator to perform numerical calculations. Some of such examples are given below.

--> 7 + 6	--> sin(0.4156)
ans =	ans =
13.	0.4037389

--> 9 / 17	--> acos(0.707)
ans =	ans =
0.5294118	0.7855492

--> 4 * 16	--> exp(2.349)
ans =	ans =
64.	10.475089

When Scilab evaluates an expression it stores the result in a variable so it can be used by subsequent statements; if the user does not explicitly assign a name, Scilab places the result in the built-in variable `ans`, whereas if the user provides an assignment the value is stored in that named variable instead, so explicit assignments are recommended when a value must be retained.

3.2 Variable assignments

Scilab allows the user to define variables and assign values to them so that they can be used later.

--> a = 3.4;	--> w = exp(0.1);
--> b = 6.78;	--> v = 2;
--> c = a + b	--> u = w .^ v
c =	u =
10.18	1.2214028

--> x = 13;	--> pi = atan(1.0)*4;
--> z = x .^ 2	--> y = sin(pi/4)
z =	y =
169.	0.7071068

3.3 Managing output display

- Appending a semicolon (;) to an expression in Scilab suppresses the output echo – the expression is still evaluated and any assignment performed, but the resulting value is not printed in the console, which is useful for reducing clutter when running scripts or storing intermediate results.
- Multiple commands may be placed on a single line by separating them with either a semicolon (;) or a comma (,). Both delimiters allow successive statements to be evaluated and any assignments to occur, but a trailing or intervening semicolon suppresses the echo of intermediate results whereas a comma causes each result to be printed. So a semi-colon is useful for keeping the console uncluttered when running sequences of commands and a comma is convenient when the user wishes to inspect intermediate outputs.

```
--> a = 55; b = 16; c = 21;      --> a = 55, b = 16, c = 21,
--> disp([a b c])              a =
55.    16.    21.              55.
                                b =
                                16.
                                c =
                                21.
                                --> disp([a b c])
                                55.    16.    21.
```

- Semicolons and commas may be mixed on the same line: any expression terminated with a semicolon (;) is evaluated (and any assignment performed) without printing its value, whereas an expression terminated with a comma (,) is evaluated and its result is echoed to the console.

```
--> x = 150; y = 213, z = 574;
y =
213.
```

- To change the font on the console window or on SciNotes, click Edit → Preferences. Then click on Fonts on the left panel.

One can then uncheck the 'Use system font' option and proceed to select the desired font typeface.

Under 'Custom font' one can then select either Console or Scinotes and choose Desktop text font under 'Fonts to use'. Finally click on Apply and then on OK to change to the desired font.

- In order to clear the console window, one can either execute the `clc` (which stands for *clear console*) or simply press the F2 key.

4 Data types

4.1 Numerical data

Although modern machines commonly offer abundant memory and a 64-bit architecture, allocating the full 64 bits to every value (for example storing the small integer 0) is often wasteful; Scilab will by default manage data representation automatically, but this can lead to inefficiencies if the widest type is used indiscriminately. Consequently, the developer may either rely on Scilab's automatic choice or explicitly declare more appropriate data types (for example single precision or integer types) where suitable; used judiciously, explicit typing reduces memory consumption and can improve execution speed, while careless use of wide types increases both memory requirements and computational overhead.

Table 1: Numeric datatypes for integers along with precision

Data type	Meaning	Range
int8	Signed 8-bit integer	$[-2^7, 2^7 - 1]$
uint8	Unsigned 8-bit integer	$[0, 2^8 - 1]$
int16	Signed 16-bit integer	$[-2^{15}, 2^{15} - 1]$
uint16	Unsigned 16-bit integer	$[0, 2^{16} - 1]$
int32	Signed 32-bit integer	$[-2^{31}, 2^{31} - 1]$
uint32	Unsigned 32-bit integer	$[0, 2^{32} - 1]$
int64	Signed 64-bit integer	$[-2^{63}, 2^{63} - 1]$
uint64	Unsigned 64-bit integer	$[0, 2^{64} - 1]$

The limits of the data types given in table 1 can be tested as follows:

--> int16(2 ¹⁵)	--> uint16(2 ¹⁶ -1)
ans = [int16]	ans = [uint16]
-32768	65535
--> int16(2 ¹⁵ -1)	--> uint16(2 ¹⁶)
ans = [int16]	ans = [uint16]
32767	0

A floating-point number is represented in the following normalized form:

$$(-1)^S M \times B^E$$

where S is the *sign*, M is the *mantissa*, B is the *base* and E is the *exponent*. Each of these components occupies specific bit-fields in memory, and the total number of bits used to represent the

value is referred to as its precision. Scilab supports several floating-point precisions (listed in table 2), and understanding how sign, mantissa and exponent are allocated helps explain rounding behaviour, range and storage cost for numerical values.

Table 2: Precisions available for floating point numbers

Precision	Bits occupied	Bytes allocated
Single	32	4
Double	64	8
Extended double	80	10
Quadruple	128	16

On a 64-bit system running Scilab, real (floating-point) variables are typically represented using 64 bits in the IEEE 754 double-precision format – hence the data type name `double` – which provides a wide dynamic range and sufficient precision for most scientific computations. This format includes the following:

- 1 bit to store the sign S , i.e. $2^1 = 2$ possible values (0 for +, 1 for –).
- 52 bits to store the mantissa M , i.e. $2^{52} = 4.5 \times 10^{15}$ possible values.
- 11 bits to store the exponent E , i.e. $2^{11} = 2048$ possible values.

Formatted display of numerical data: Scilab provides two complementary ways to control how numeric results appear: a *global display format* that governs the default printing of numbers at the prompt, and *explicit format strings* for precise control when producing output to the console or to files. Use the global format for convenient, interactive inspection and use the explicit format functions when exact field widths, number of digits or layout are required.

1. *Global display format:* The `format` function sets the default way Scilab prints decimal numbers at the prompt and in functions such as `disp`. It supports a *mode* (for example a variable/default ‘v’ mode or an exponential ‘e’ mode) and a *width* (the number of output characters used, all included: sign of the mantissa, its digits, decimal separator, exponent symbol, sign and digits of the exponent).

The call forms include `format(mode)`, `format(width)`, and `format(mode, width)`, and `format()` returns the current settings. Changing the global format is convenient for interactive work when one wants more or fewer significant digits without rewriting print statements.

```
--> pi = 22/7;

--> disp(pi)
3.1428571

--> format("v",6)
```

```
--> disp(pi)
3.143

--> format("e",10)

--> disp(pi)
3.143D+00
```

2. *Explicit format strings*: For formatted output (tables, reports, iteration logs, files), Scilab implements C-style conversion specifiers via `mprintf` (console), `mfprintf` (file) and `msprintf` (return a string). These functions accept format strings containing conversion specifiers such as `%f`, `%e`, `%g`, and allow specifying field width, precision and numbered placeholders.

4.2 Boolean data

Boolean data is assigned to variables using either `%T` (for *true*) or `%F` (for *false*).

```
--> a = %T
a =
T
```

Boolean variables must be operated using boolean operators which are the following:

- NOT – represented by the symbolic operator `~`

```
--> b = ~a
b =
F
```

- AND – represented by the symbolic operator `&`

```
--> a & b
ans =
F
```

- OR – represented by the symbolic operator `|`

```
--> a | b
ans =
T
```

4.3 Strings

Alphabets along with special characters are treated as string data types. String variable and values can be concatenated using the + operator.

```
--> s1 = "Hello";

--> s2 = "world";

--> s = s1 + s2
s =
    "Helloworld"

--> s = s1 + ", " + s2 + "!"
s =
    "Hello, world!"
```

The built-in function `string()` converts a given value to string data type. As shown in the following example, the numerical value 2 is converted to a string data type when the `string()` function is applied to it.

```
--> a = 2
a =
    2.

--> s = string(a)
s =
    "2"
```

One can now concatenate the string format of “2” with another number using the + operator.

```
--> t = string(5)
t =
    "5"

--> s + t
ans =
    "25"
```

4.4 Complex numbers

Scilab supports complex numbers as a separate numeric type, using the imaginary unit %i (also available as the shorthand 1i or 1j) so that expressions such as $2 + 3\%i$ or $2 + 3i$ represent the complex number $2 + 3i$.

```
--> %i                                --> 2 + 3*%i
%i =                                  ans =
0. + i                                2. + 3.i

--> 1i                                --> 2 + 3i
ans =                                 ans =
0. + i                                2. + 3.i
```

Complex numbers can also be created from separate real and imaginary parts with the constructor `complex(a, b)`, which is particularly convenient for arrays.

```
--> z = complex(4, -5)
z =
4. - 5.i
```

The real and imaginary parts of a complex number can be extracted using the `real()` and `imag()` functions respectively.

```
--> real(z)                            --> imag(z)
ans =                                  ans =
4.                                    -5.
```

5 Insert section title

Insert section content.